

مشروع مادة البرمجة التفرعية

الإختصاص: الشبكات و النظم الحاسوبية

الطلاب:

عمر ابراهيم

يوسف الشيخ علي

سلام ناصر

تسنيم عبد الجليل

1. نظرة عامة على المشروع

هذا المشروع عبارة عن واجهة خلفية (backend) وهمية لمنصة تجارة إلكترونية مبنية باستخدام Spring Boot، ويُستخدم لدراسة التزامن (concurrency)، وإدارة الموارد، وسلوك النشر (deployment) تحت ضغط الأحمال. تشمل العمليات التجارية الأساسية تسجيل المستخدمين وتسجيل الدخول، إدارة المنتجات والفئات، التعامل مع سلة المشتريات، إتمام الطلب (checkout)، سجل الطلبات، سجل المعاملات، والتقارير الإدارية. تم تقييم النظام عمداً تحت الضغط باستخدام JMeter بهدف كشف نقاط الضعف الهندسية قبل تطبيق التحسينات المستهدفة.

المتطلبات التي يغطيها هذا التقرير هي:

1. التعديل المتزامن الآمن لمورد مشترك
2. إدارة الموارد والتحكم في السعة
3. المعالجة غير المتزامنة Asynchronous Queues
4. معالجة البيانات الضخمة على دفعات (Batch Processing)
5. محاكاة توزيع الحمل (load distribution) عبر خوادم متعددة

في حالتنا، يُعد مخزون المنتجات أثناء إتمام الطلب هو المورد المشترك الأكثر أهمية. بمجرد حل هذه المشكلة، كانت المشكلة التالية هي الحد من مقدار العمل المتوازي الذي يقبله الخادم في وقت واحد. أخيراً، تم توسيع التطبيق من نموذج نشر أحادي العقدة (single-node) إلى بنية متعددة النسخ (multi-instance) مع موازنة الأحمال (load-balanced).

2. المنهجية الهندسية

تم استخدام نفس المنهجية عبر المتطلبات:

1. البدء من أبسط نسخة تعمل من التطبيق.
2. تحديد نمط الفشل أو مشكلة قابلية التوسع (scalability).
3. إعادة إنتاج تلك المشكلة باستخدام JMeter والفحص المباشر لقاعدة البيانات.
4. مقارنة الحلول الهندسية المتعددة.
5. اختيار الحل الأنسب لحجم المشروع، وتكلفة التنفيذ، والوضوح، وقابلية العرض.
6. تنفيذ الحل بطريقة يمكن اختبارها وشرحها.
7. التحقق من صحة النتيجة باستخدام كل من الاختبارات الآلية (automated tests) واختبارات التحميل (load tests).

تم اختبار هذا النهج لأنه لا ينتج فقط تنفيذاً قابلاً للعمل، بل يوفر أيضاً أساليب هندسية يمكن الاعتماد عليها في سوق العمل.

3. المتطلب 1: التعديل المتزامن الآمن لنفس المورد

3.1 بيان المشكلة

يجب أن يسمح التطبيق لعدة مستخدمين بالتفاعل مع نفس المنتج في نفس الوقت دون إفساد بيانات المخزون. الحالة الحرجة هنا هي إتمام الطلب: إذا حاول مستخدمان أو أكثر شراء آخر وحدة متبقية من منتج ما في وقت واحد، يجب على النظام منع البيع الزائد عن المخزون (overselling). في التنفيذ الأساسي (baseline) غير الآمن، كان تدفق إتمام الطلب يجري فحصاً للمخزون وتحديثاً له دون حماية قوية للترامن. أدى هذا إلى السماح لمعاملات متعددة (transactions) بقراءة نفس قيمة المخزون قبل أن تقوم أي منها باعتماد (commit) تحديثها.

3.2 السلوك الأساسي قبل الإصلاح

قبل تنفيذ المتطلب الأول، أظهر التطبيق حالة تسابق (race condition) كلاسيكية. تحت اختبار التنافس (contention test) حيث حاول العديد من المستخدمين المتزامنين شراء منتج مخزونه يساوي 1، سمح النظام بأكثر من عملية إتمام طلب ناجحة.

المعنى الهندسي لهذه النتيجة مهم جداً:

- تمكن أكثر من مستخدم من شراء نفس العنصر الأخير.
- لم يكن فحص المخزون وتحديثه عمليتين (atomic) من منظور التزامن.
- قام النظام ببيع مخزون زائد تحت ظروف التنافس.

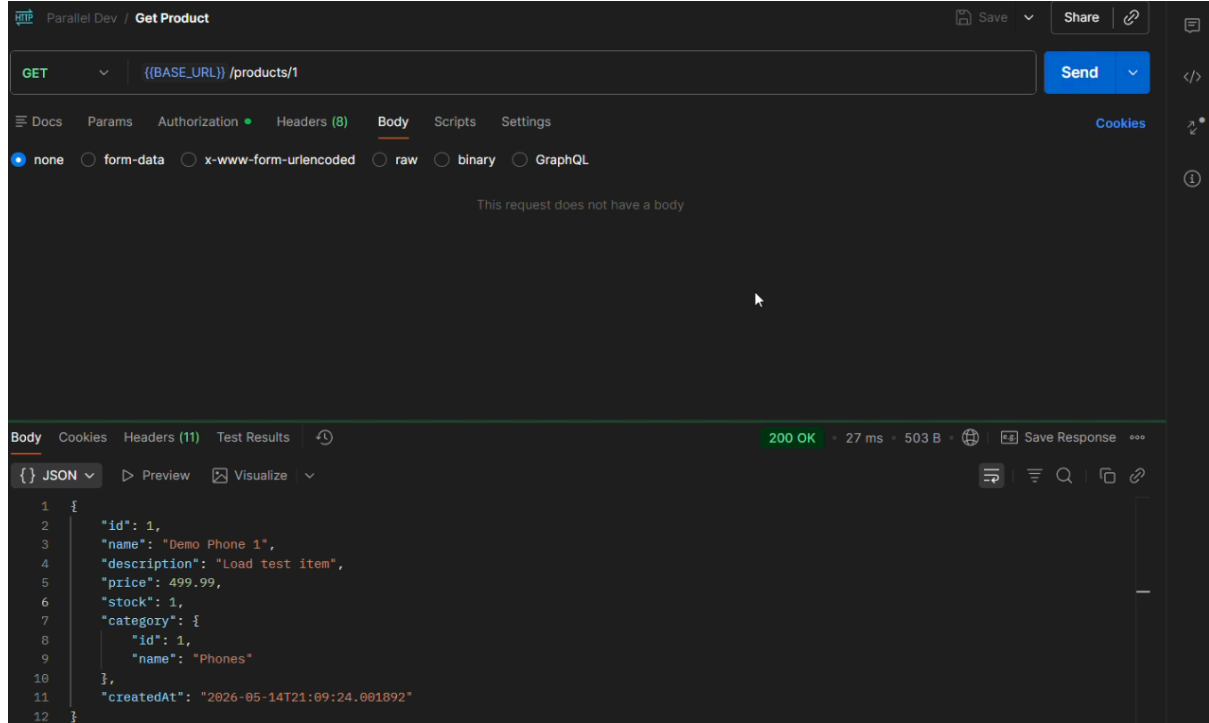
لم تكن المشكلة الأساسية نظرية فقط؛ بل كانت قابلة للملاحظة من خلال JMeter وحالة قاعدة البيانات.

ملخص السلوك السابق

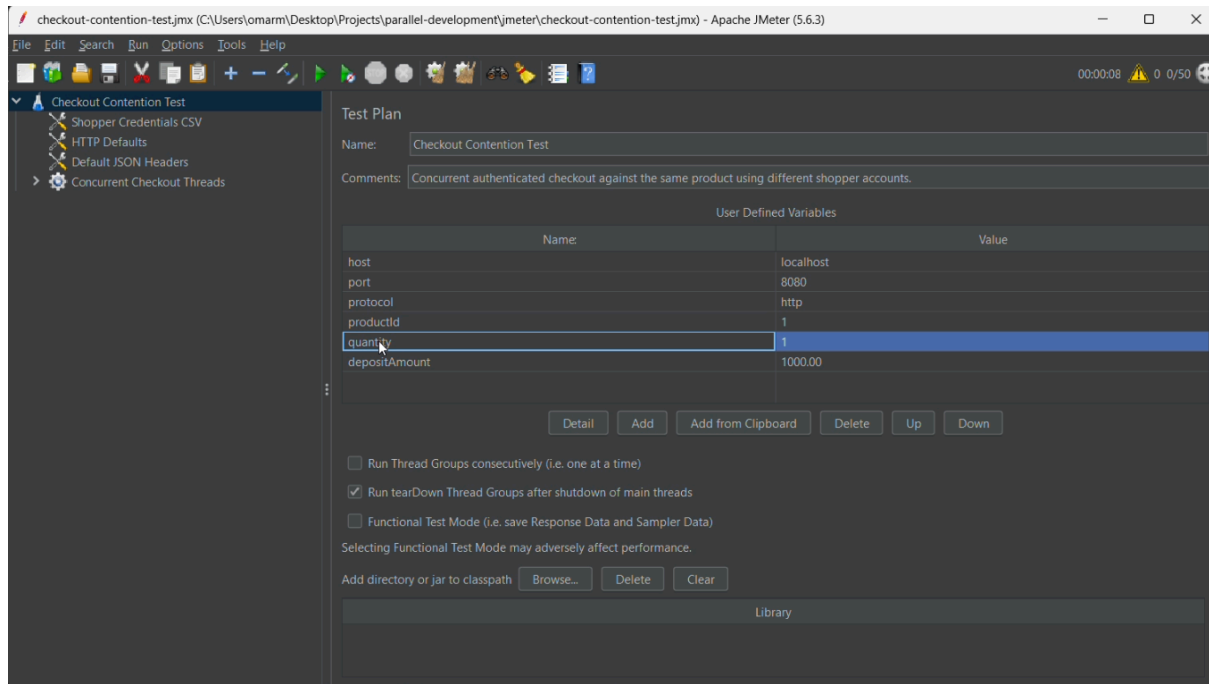
- كان من الممكن بيع مخزون مشترك بشكل زائد
- كان من الممكن إجراء عدة عمليات شراء ناجحة لوحدة واحدة متبقية
- لم تكن القيمة النهائية للمخزون وحدها مؤشراً كافياً على الصحة
- الدليل الحقيقي كان أن المشتريات الناجحة تجاوزت المخزون المتاح

3.3 الإثباتات بالصور قبل حل المشكلة

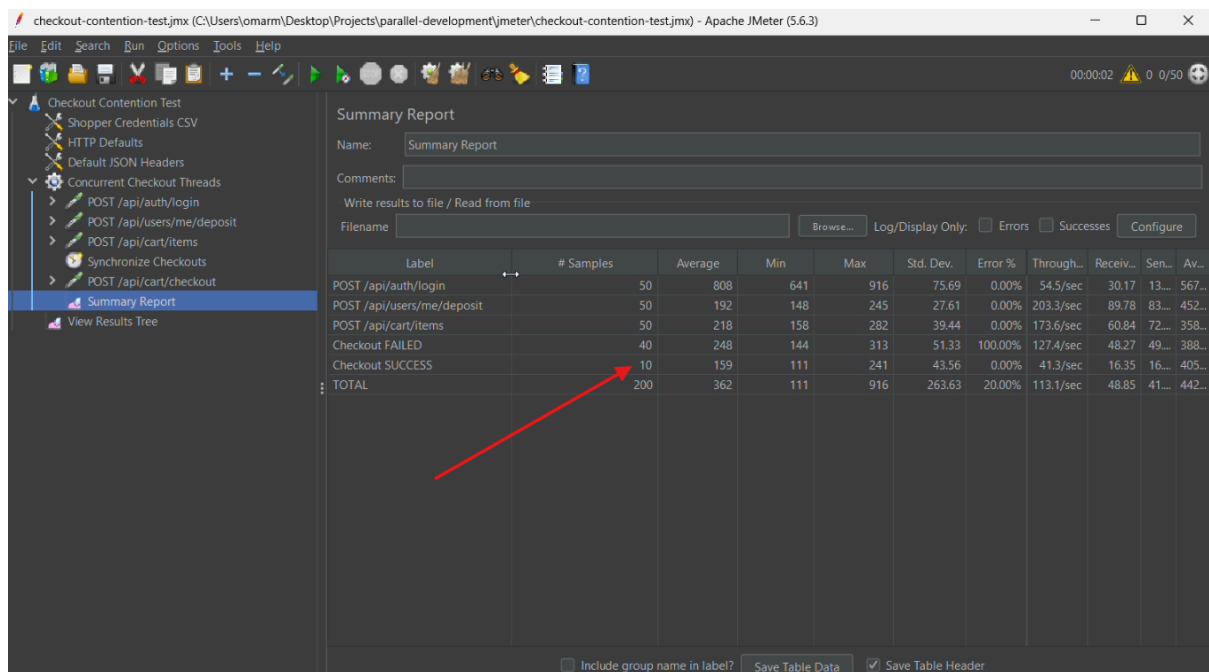
صورة تظهر وجود عنصر واحد في المخزون



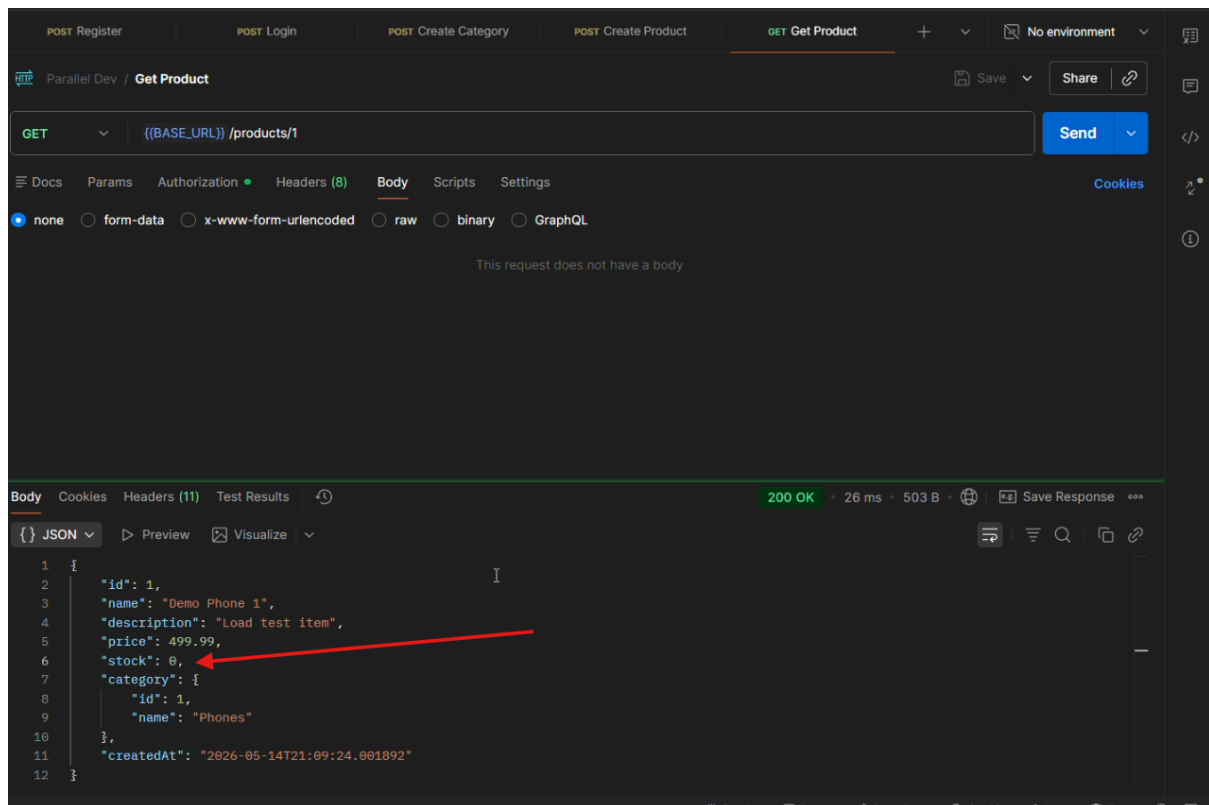
صورة تظهر اعدادات Jmeter



صورة تظهر 10 عمليات شراء ناجحة رغم وجود منتج واحد متاح



صورة تظهر أن العدد المتاح من المنتج أصبح صفر



صورة تظهر وجود 10 عمليات شراء ناجحة في الداتابيز

```
ecommerce=# SELECT * FROM orders;
```

id	created_at	status	total_price	user_id
2	2026-05-14 21:10:52.132948	SUCCESS	499.99	43
9	2026-05-14 21:10:52.132948	SUCCESS	499.99	35
10	2026-05-14 21:10:52.132948	SUCCESS	499.99	23
7	2026-05-14 21:10:52.132948	SUCCESS	499.99	48
8	2026-05-14 21:10:52.132948	SUCCESS	499.99	2
6	2026-05-14 21:10:52.133458	SUCCESS	499.99	4
1	2026-05-14 21:10:52.133458	SUCCESS	499.99	42
5	2026-05-14 21:10:52.132948	SUCCESS	499.99	36
4	2026-05-14 21:10:52.132948	SUCCESS	499.99	33
3	2026-05-14 21:10:52.132948	SUCCESS	499.99	27

(10 rows)

```
ecommerce=# SELECT * FROM products;
```

id	created_at	description	name	price	stock	category_id
1	2026-05-14 21:09:24.001892	Load test item	Demo Phone 1	499.99	0	1

(1 row)

3.4 الحلول البديلة المقترحة

تم النظر في عدة نهج صالحة:

الخيار أ: القفل (Pessimistic locking)

هذا النهج سيقفل صف المنتج في قاعدة البيانات أثناء عملية الدفع باستخدام آلية من نوع `SELECT ... FOR UPDATE`.

المزايا:

- ضمان صحة قوي جداً
- سهل الفهم والتحليل في سيناريوهات التنافس العالي

العيوب:

- يحظر المعاملات المتنافسة
- يقلل من التزامن
- يمكن أن يؤدي إلى أوقات انتظار أطول وخطر الاختناق المروحي (deadlock) إذا لم يتم تصميمه بعناية

الخيار ب: القفل الموزع (Distributed locking)

هذا النهج يستخدم مدير أقفال خارجي مثل Redis لعمل تسلسل (serialize) للوصول إلى تحديثات المخزون.

المزايا:

- مفيد في البيانات الموزعة متعددة العقد
- يمكنه تنسيق الوصول إلى المخزون عبر عدة نسخ (instances)

العيوب:

- يضيف تعقيداً للبنية التحتية الخارجية
- من الصعب تبريره لمشروع Spring Boot صغير نسبياً
- أجزاء متحركة أكثر مما هو مطلوب للمرحلة الحالية

الخيار ج: إدراج جميع عمليات الشراء في طابور من خلال عامل (worker) واحد

هذا النهج سيرسل جميع عمليات الشراء إلى طابور ويقوم بمعالجتها بشكل تسلسلي.

المزايا:

- ضمان قوي للتسلسل
- آمن جداً لتحديثات المخزون شديدة النشاط (hotspots)

العيوب:

- يضيف زمن انتقال (latency) لعملية الدفع
- يغير نموذج الطلب/الاستجابة بشكل كبير
- أكثر ملاءمة لمتطلبات المعالجة غير المتزامنة (asynchronous) اللاحقة

الخيار د: القفل (Optimistic locking) مع إعادة المحاولة

يسمح هذا النهج للمعاملات المتزامنة بالمضي قدماً بشكل طبيعي، ولكنه يكتشف التعارضات في وقت الاعتماد/التحديث (commit/update) باستخدام إدارة الإصدارات (versioning). تتم إعادة محاولة المعاملات المتعارضة لعدد محدود من المرات أو يتم رفضها بشكل سلس.

المزايا:

- أداء جيد عندما يكون التنافس معتدلاً
- يتناسب بشكل أصلي مع JPA/Hibernate
- أبسط من الأقفال الموزعة أو العمال المتسلسلين (serialized workers)
- سهل الشرح والإثبات في مشروع دراسي

العيوب:

- تحت التنافس الشديد، يجب أن تفشل بعض الطلبات أو تعيد المحاولة
- يتطلب معالجة صريحة للتعارضات

3.5 الحل المختار والمبررات

لقد اخترنا القفل (Optimistic locking) مع إعادة المحاولة. كان هذا هو الحل الأنسب للأسباب التالية:

- يستخدم المشروع بالفعل JPA Spring Data و Hibernate، اللذين يدعمان القفل (Optimistic locking) مع إعادة المحاولة بشكل طبيعي.
- التطبيق لا يزال مضغوطاً نسبياً، لذا فإن إضافة خدمة قفل موزعة كان سيؤدي إلى تعقيد غير ضروري.
- تعارضات إتمام الطلب مقتصرة على عدد قليل من الصفوف، خاصة **Product** (المنتج).
- الحل سهل التحقق منه تجريبياً: بعد الإصلاح، يجب أن يسمح منتج بمخزون 1 بعملية شراء واحدة ناجحة فقط.
- يعلم نمط تزامن واقعي ومهم دون المبالغة في هندسة النظام (overengineering).

3.6 تفاصيل التنفيذ

يستخدم التنفيذ آليتين رئيسيتين:

1. القفل Optimistic Locking عبر JPA على كيان المنتج (product entity)

يحتوي كيان المنتج (**Product**) الآن على حقل إصدار:

- الملف: `src/main/java/com/ecommerce/ecommerce/product/Product.java`
- الآلية: `Version@`

هذا يجعل Hibernate يقوم بتضمين الإصدار في عمليات التحديث. إذا قامت معاملة أخرى بتغيير نفس الصف أولاً، تكتشف المعاملة الثانية التعارض بدلاً من الكتابة فوق البيانات بصمت.

2. إعادة المحاولة والتعافي في عملية إتمام الطلب

تتم معالجة منطق إتمام الطلب في:

- الملف: `src/main/java/com/ecommerce/ecommerce/cart/CartService.java`

خيارات التصميم الرئيسية:

- الطريقة `Transactional@checkout(...)` محددة بـ `@Transactional`
 - وهي أيضاً `Retryable@` (قابلة لإعادة المحاولة) لاستثناء `OptimisticLockingFailureException`
 - عمليات إعادة المحاولة مقتصرة على عدد صغير من المحاولات
 - يتم استدعاء `productRepository.flush()` قبل إنشاء الآثار الجانبية النهائية للطلب حتى تظهر تعارضات المخزون مبكراً
 - إذا استمر فشل محاولات إعادة المحاولة، تقوم `@Recover` بإلقاء استثناء `InventoryConflictException`
- هذا يعني أن التطبيق لا يخفي التعارضات. بدلاً من ذلك، فإنه إما ينجح بسلاسة أو يُرجع استجابة تعارض محكومة.

3.7 لماذا ينجح هذا التنفيذ

سلوك التزامن المهم هو:

1. قد يبدأ مستخدمان في إتمام الطلب في نفس الوقت تقريباً.
2. قد يرى كلاهما في البداية نفس قيمة المخزون.
3. تقوم المعاملة الأولى بتحديث المنتج وزيادة رقم الإصدار.
4. تحاول المعاملة الثانية التحديث باستخدام الإصدار القديم.
5. يكشف Hibernate عدم تطابق الإصدار ويلقي بخطأ فشل القفل المتفائل.
6. يقوم Spring Retry بإعادة محاولة العملية أو يفشل بشكل سلس.

هذا يقضي على مشكلة التحديثات المفقودة الصامتة (silent lost updates) ويمنع البيع الزائد.

3.8 التحقق والإثبات

تم إجراء التحقق بطريقتين:

أ.

اختبار التنافس عبر JMeter

تم إنشاء منتج واحد بمخزون يساوي 1، وحاول عدة مستخدمين متزامنين إتمام الطلب في نفس الوقت.

السلوك الآمن المتوقع:

- عملية شراء واحدة ناجحة بالضبط
- البقية تفشل بسبب تعارض المخزون أو عدم التوفر

ب.

اختبار تكامل التزامن الآلي (Automated concurrency integration test)

الملف:

- `src/test/java/com/ecommerce/ecommerce/cart/CartServiceConcurrencyIntegrationTest.java`

هذا الاختبار ينشئ:

- منتجاً واحداً بمخزون 1
- متسوقين متعددين لديهم نفس المنتج في عرباتهم
- محاولات دفع متزامنة

ويتحقق من:

- عملية شراء واحدة ناجحة بالضبط
- المخزون النهائي هو 0
- يوجد طلب واحد ناجح بالضبط
- توجد معاملة شراء واحدة بالضبط

3.9 السلوك بعد الإصلاح

بعد تنفيذ المتطلب الأول:

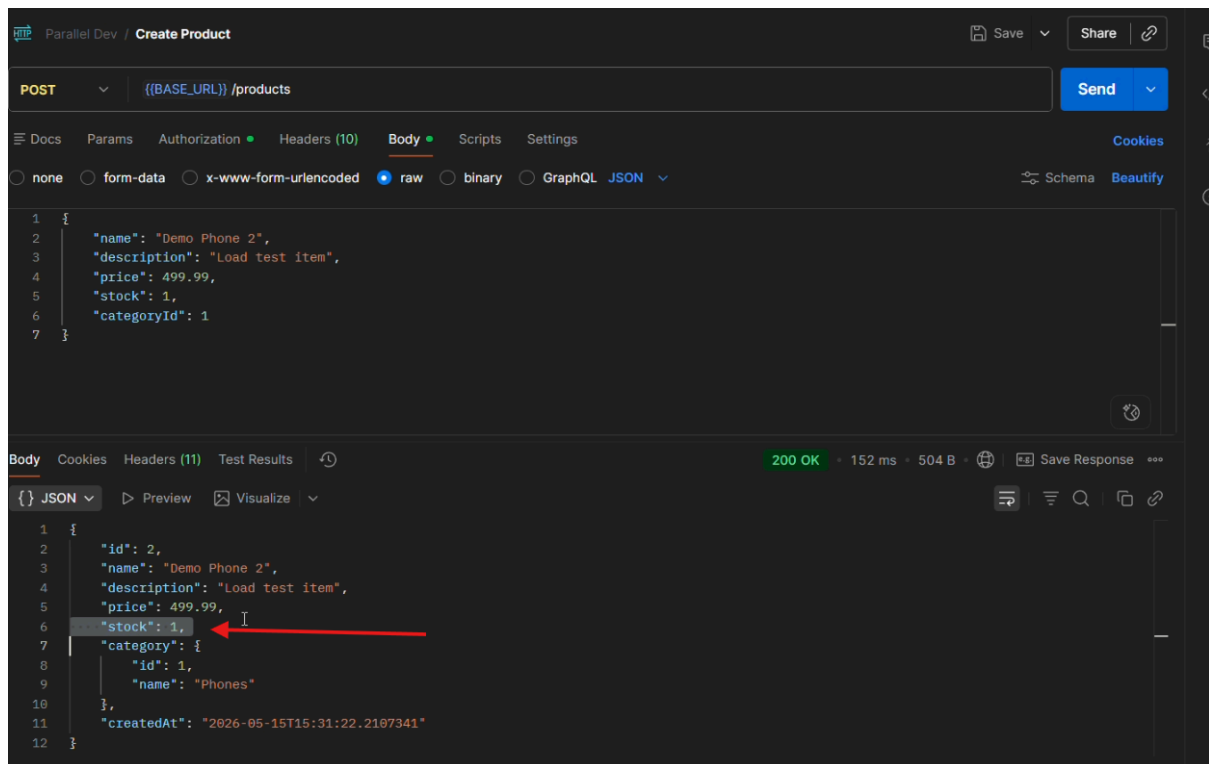
- لم يعد يحدث بيع زائد في السيناريو المُختبر
- تم السماح بعملية شراء واحدة فقط لمنتج بمخزون 1
- أصبحت التعارضات صريحة بدلاً من إفساد المخزون بصمت
- حافظ التطبيق على صحة البيانات تحت عبء إتمام الطلب المتزامن.

ملخص السلوك بعد الإصلاح

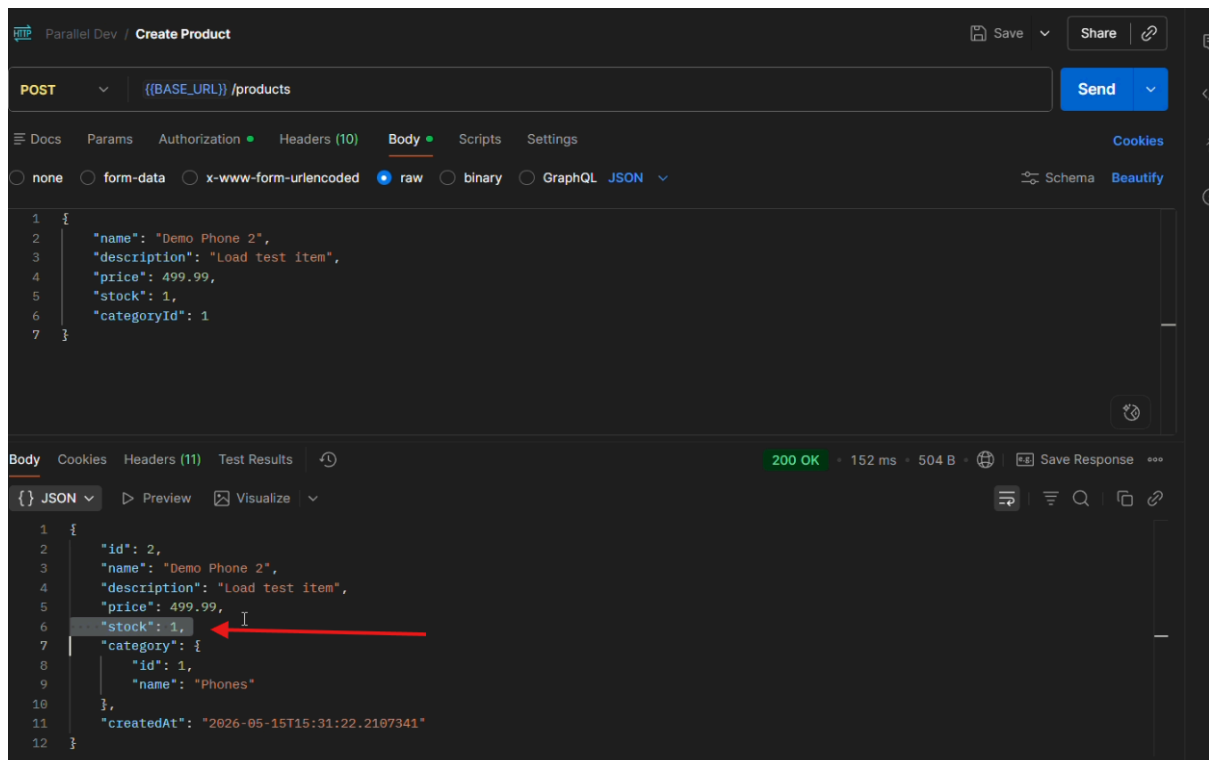
- تم الحفاظ على صحة المخزون
- أصبحت تحديثات المخزون المشترك آمنة من التعارض
- تمت معالجة حالة التسابق (race condition) بدلاً من تجاهلها
- أصبحت إخفاقات إتمام الطلب في ظل التنافس محكومة بدلاً من إفساد حالة النظام

3.10 الإثباتات بالصور بعد حل المشكلة

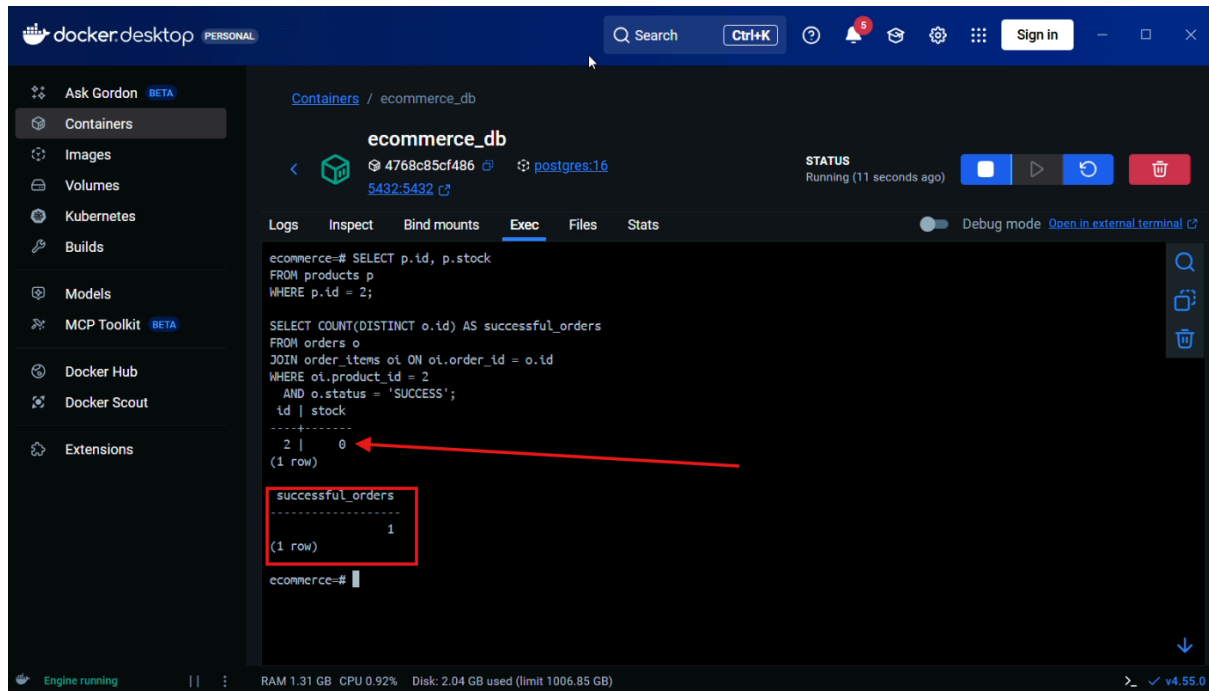
صورة تظهر وجود قطعة واحدة في المخزون



صورة تظهر وجود عملية شراء واحدة ناجحة فقط



صورة تظهر وجود عملية شراء واحدة ناجحة فقط من الداتايبز



4. المتطلب 2: إدارة الموارد والتحكم في السعة

4.1 بيان المشكلة

بمجرد حل المتطلب الأول، تحسن الأداء في ظل التزامن، لكن التطبيق كان لا يزال بحاجة إلى حماية ضد قبول قدر كبير من العمل في وقت واحد.

إذا قبل الخادم طلبات أكثر مما يمكنه معالجته بكفاءة، فقد يواجه:

- نمواً مفرطاً في (threads) المعالجة
- استنفاد تجمع اتصالات قاعدة البيانات (connection pool)
- ارتفاعات مفاجئة في زمن الاستجابة (latency spikes)
- انتهاء مهلة الطلبات (timeouts)
- فشل في الاتصال على مستوى النقل (transport-level)
- سلوكاً غير مستقر تحت حركة المرور المفاجئة (burst traffic)

لذلك، ركز المتطلب الثاني على الحد من مقدار العمل المتزامن المسموح بدخوله إلى النظام.

4.2 السلوك الأساسي قبل الإصلاح

قبل هذه المرحلة، كان التطبيق يعتمد بشكل أساسي على السلوك الافتراضي لكونتينر السيرفلت (servlet container) وتجمع قاعدة البيانات. تحت التحميل العدواني (Aggressive Testing) باستخدام JMeter، خلق هذا عدة مخاطر:

- يمكن أن يدخل عدد كبير جداً من الطلبات إلى النظام في وقت واحد
- لم تتم إدارة الحمل الزائد (overload) بشكل مركزي
- الإخفاقات، عند حدوثها، لم تكن دائماً متعمدة أو محكمة
- تشبع الموارد كان يمكن أن يحدث في مستويات أعمق من المكس (stack) بدلاً من نقطة الدخول

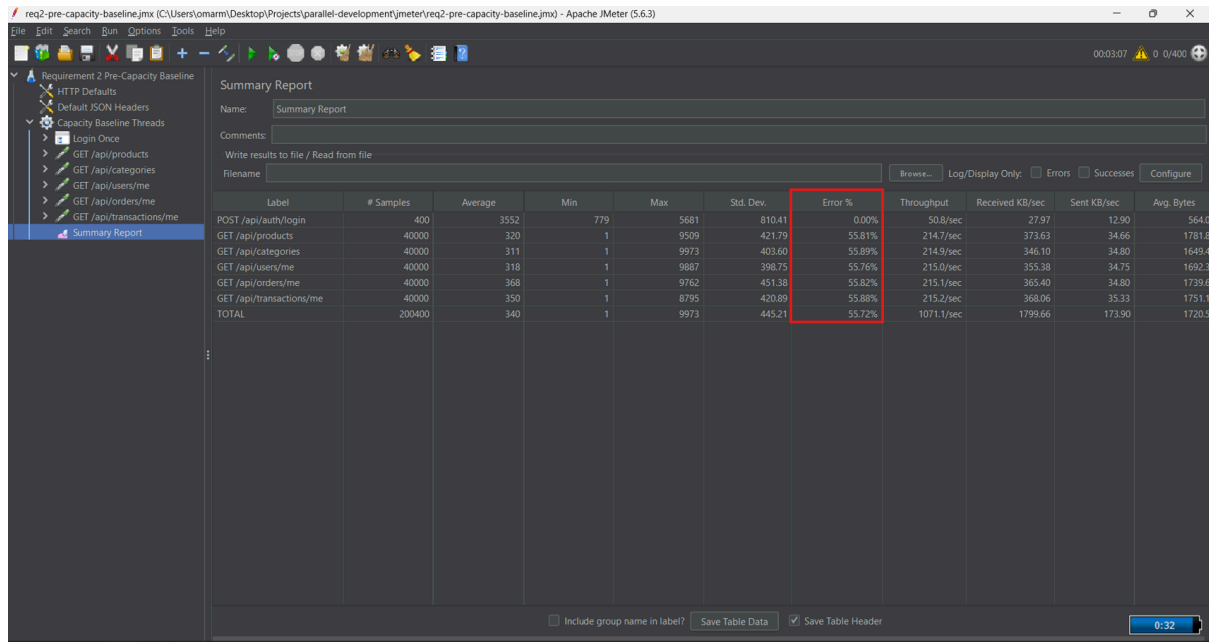
هذا تمييز هندسي مهم. النظام المستقر يجب ألا يفشل "في مكان ما لاحقاً". يجب أن يرفض العمل الزائد مبكراً وبشكل يمكن التنبؤ به.

ملخص السلوك السابق

- لا يوجد تحكم عام صريح في قبول الطلبات
- كانت حدود الموارد ضمنية بدلاً من إدارتها مركزياً
- لم تكن معالجة الحمل الزائد محكمة بقوة
- دفعات الطلبات الكبيرة يمكن أن تنتج سلوكاً غير مستقر

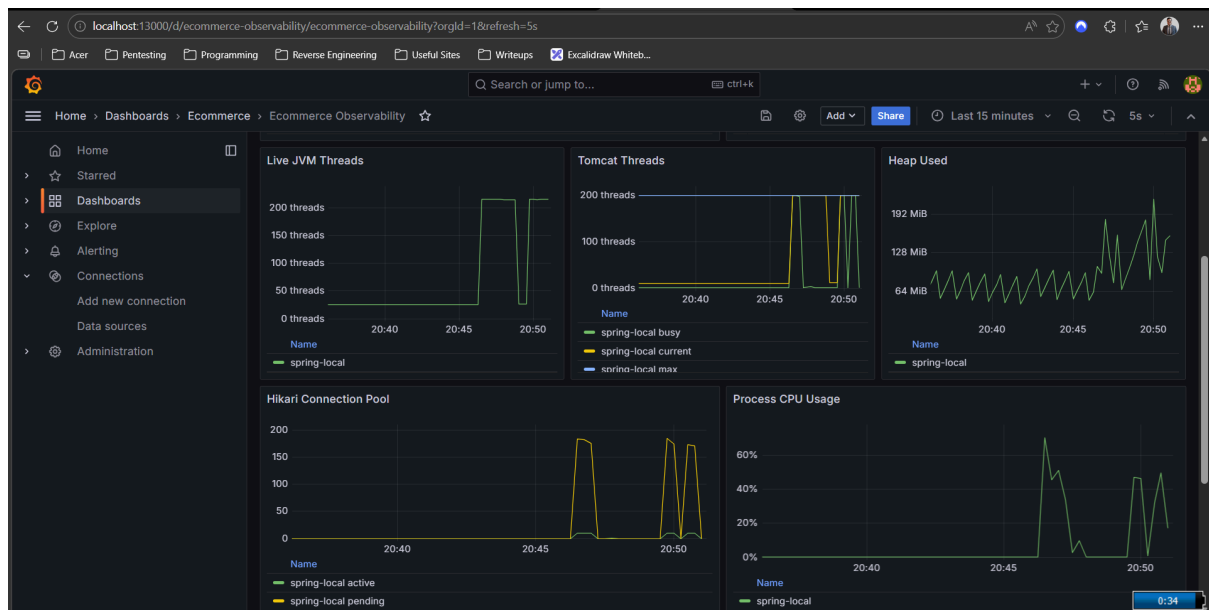
4.3 الإثباتات بالصور قبل حل المشكلة

صورة تظهر أن نسبة الأخطاء وصلت إلى 55% قبل تطبيق إدارة الموارد



Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
POST /api/auth/login	400	3552	779	5681	810.41	0.00%	50.8/sec	27.97	12.90	564.0
GET /api/products	40000	320	1	9509	421.79	55.81%	214.7/sec	373.63	34.66	1781.8
GET /api/categories	40000	311	1	9973	403.60	55.89%	214.9/sec	346.10	34.80	1649.4
GET /api/users/me	40000	318	1	9887	398.75	55.76%	215.0/sec	355.38	34.75	1692.3
GET /api/orders/me	40000	368	1	9762	451.38	55.82%	215.1/sec	365.40	34.80	1739.6
GET /api/transactions/me	40000	350	1	8795	420.89	55.88%	215.2/sec	368.06	35.33	1751.1
TOTAL	200400	340	1	9973	445.21	55.72%	1071.1/sec	1799.66	173.90	1720.5

صورة تظهر إستهلاك ضخمة للموارد عن طريق واجهة غرافانا



صورة تظهر أنواع بعض الأخطاء السابقة في Jmeter

Errors			
Type of error	Number of errors	% in errors	% in all samples
Non HTTP response code: org.apache.http.conn.HttpHostConnectException/Non HTTP response message: Connect to localhost:8080 [localhost/127.0.0.1, localhost/0.0.0.0:0.0.0.1] failed: Connection refused: getsockopt	4410	100.00%	11.71%

Top 5 Errors by sampler										
Sample	#Samples	#Errors	Error	#Errors	Error	#Errors	Error	#Errors	Error	#Errors
Total	37650	4410	Non HTTP response code: org.apache.http.conn.HttpHostConnectException/Non HTTP response message: Connect to localhost:8080 [localhost/127.0.0.1, localhost/0.0.0.0:0.0.0.1] failed: Connection refused: getsockopt	4410						
GET /api/categories	7500	870	Non HTTP response code: org.apache.http.conn.HttpHostConnectException/Non HTTP response message: Connect to localhost:8080 [localhost/127.0.0.1, localhost/0.0.0.0:0.0.0.1] failed: Connection refused: getsockopt	870						
GET /api/orders/me	7500	890	Non HTTP response code: org.apache.http.conn.HttpHostConnectException/Non HTTP response message: Connect to localhost:8080	890						

4.4 الحلول البديلة المقترحة

تم النظر في العديد من استراتيجيات إدارة الموارد:

الخيار أ: ضبط Tomcat و HikariCP فقط

سيعتمد هذا النهج فقط على حدود خيوط الحاوية وإعدادات تجمع قاعدة البيانات.

المزايا:

- أكواد قليلة جداً
- ضبط أساسي مفيد

العيوب:

- لا يخلق سياسة واضحة للحمل الزائد على مستوى التطبيق
- قد لا تزال الطلبات تفشل في مرحلة متأخرة بدلاً من مبكرة
- من الصعب شرحها كآلية هندسية مخصصة

الخيار ب: استخدام معدل (rate limiter) خارجي على الـ reverse proxy فقط

سيفرض هذا النهج قيوداً في Nginx أو أي وكيل (proxy) آخر.

المزايا:

- حماية فعالة على مستوى الحافة (edge-level)
- شائع في بيئات الإنتاج (production)

العيوب:

- لا يوضح الآلية داخل التطبيق نفسه
- أقل توافقاً مع الهدف المتمثل في تحسين تطبيق Spring

الخيار ج: العزل لكل خدمة (Per-service bulkheads)

هذا النهج سيطبق ضوابط تزامن منفصلة على طرق (methods) خدمات معينة باستخدام أنماط مثل الـ bulkheads (العوازل).

المزايا:

- تحكم دقيق (Fine-grained)
- مفيد عندما تتطلب خدمات مختلفة حدوداً مختلفة

العيوب:

- أكثر تعقيداً في التصميم والضبط
- أقل وضوحاً في العرض خلال مرحلة أولى للتحكم في الموارد على مستوى النظام

الخيار د: التحكم في قبول الطلبات على مستوى النظام

يطبق هذا النهج حارساً على مستوى التطبيق بأكمله للترامن (concurrency guard) على مستوى طلبات الويب. إذا كان هناك عدد كبير جداً من الطلبات قيد المعالجة (in flight)، تنتظر الطلبات الجديدة لفترة وجيزة أو تُرفض بخطأ 429 Too Many Requests.

المزايا:

- واضح وسهل الشرح
- يحمي طبقة الـ API بأكملها، وليس طريقة (method) واحدة فقط
- يوفر قصة مباشرة للمقارنة بين قبل وبعد

- ينتج سلوكاً يمكن التنبؤ به للحمل الزائد

العيوب:

- أقل دقة من استراتيجية التمييز الكاملة لكل نقطة نهاية (endpoint)
- يتطلب ضبطاً لتجنب الرفض المفرط أو الحماية غير الكافية

4.5 الحل المختار والمبررات

لقد اخترنا التحكم في قبول الطلبات على مستوى النظام، مقترناً بضبط حاوية السيرفلت وتجمع قاعدة البيانات. تم اختيار هذا بسبب:

- آلية مركزية واحدة أسهل في التحقق من صحتها من عدة خناقات (throttles) مبعثرة
- تحمي التطبيق قبل أن تصبح الطبقات الأعمق محملة بشكل زائد
- ينتج عنها نتيجة هندسية واضحة: يظل العمل المقبول مقيداً بحدود
- تتكامل بشكل طبيعي مع أدوات المراقبة Prometheus/Grafana

4.6 تفاصيل التنفيذ

يحتوي التنفيذ على أربعة أجزاء رئيسية:

1. حارس سعة الطلبات (Request-capacity guard)

الملف:

- `src/main/java/com/ecommerce/ecommerce/capacity/RequestCapacityGuard.java`

يستخدم هذا المكون كائن `Semaphore` عادل للتحكم في عدد طلبات الـ API التي يمكن أن تكون قيد المعالجة في نفس الوقت. الميزات الرئيسية:

- الحد الأقصى لعدد الطلبات المتزامنة
- انتظار قصير قبل الرفض
- مقاييس (metrics) للطلبات قيد المعالجة، المتاحة، الحد الأقصى، والمرفوضة

هذا يجعل سياسة القبول صريحة وقابلة للقياس.

2. مرشح سعة الطلبات (Request-capacity filter)

الملف:

- `src/main/java/com/ecommerce/ecommerce/capacity/RequestCapacityFilter.java`

هذا المرشح هو نقطة التنفيذ. السلوك:

- يحمي مسارات `/api/`
- يستثني:
 - `/api/auth/`
 - `/actuator/`
 - `/h2-console/`

- error/
- OPTIONS
- يحاول الحصول على تصريح (permit) من الحارس
- ينتظر لفترة وجيزة حتى الحد المكوّن
- إذا كان التصريح غير متاح، يرجع:
- رمز HTTP 429
- رسالة خطأ بصيغة JSON
- ترويسة Retry-After: 1

هذا يعني أن التطبيق يرفض العمل الزائد مبكراً بدلاً من الإفراط في إلزام الخيوط والاتصالات.

3. دمج المرشح مع الأمن (Security)

الملف:

- `src/main/java/com/ecommerce/ecommerce/security/SecurityConfig.java`

يتم وضع مرشح سعة الطلبات قبل نقطة الدخول الرئيسية لسلسلة مرشحات المصادقة. هذا مهم لأنه يمنع التطبيق من إهدار العمل في تحليل الـ JWT، وتوجيه وحدة التحكم (controller dispatch)، والوصول لقاعدة البيانات عندما تكون السعة مستنفدة بالفعل.

4. ضبط وقت التشغيل في خصائص التطبيق

الملف:

- `src/main/resources/application.properties`

الإعدادات الرئيسية:

- `server.tomcat.threads.max=100`
- `server.tomcat.accept-count=200`
- `spring.datasource.hikari.maximum-pool-size=20`
- `spring.datasource.hikari.minimum-idle=10`
- `spring.datasource.hikari.connection-timeout=3000`
- `app.capacity.max-concurrent-requests=60`
- `app.capacity.max-wait-ms=200`

هذه القيم تعمل معاً:

- عدد خيوط Tomcat مقيد بحد
- تم تكبير طابور القبول في Tomcat لتقليل الرفض العابر للاتصالات خلال الدفعات القصيرة
- اتصالات قاعدة البيانات مقيدة
- قبول الطلبات مقيد بشكل صريح
- يتم إما قبول الطلبات الزائدة ضمن تأخير قصير أو رفضها بسلاسة

4.7 المراقبة وقابلية الملاحظة

تم دعم المتطلب الثاني بواسطة حزمة مراقبة تعتمد على:

- Spring Actuator
- Micrometer
- Prometheus
- Grafana

سمح هذا بمراقبة:

- إنتاجية الطلبات (throughput)
- زمن انتقال الطلب (latency)
- استخدام خيوط Tomcat
- نشاط HikariCP والاتصالات المعلقة
- مقاييس السعة المخصصة

أهم المقاييس المخصصة التي تم إدخالها في هذه المرحلة كانت:

- `app.capacity.requests.in_flight`
- `app.capacity.requests.available`
- `app.capacity.requests.max`
- `app.capacity.requests.rejected`

كانت طبقة المراقبة هذه بالغة الأهمية لأن المتطلب الثاني يتعلق بسلوك الموارد، وليس فقط بالصحة الوظيفية.

4.8 التحسين الهندسي التكراري

تطلب المتطلب الثاني ضبطاً بدلاً من تغيير واحد لمرة واحدة. أثناء الاختبار، ظهرت فئة واحدة من الإخفاقات:

- رفض الاتصال (`Connection refused`) على مستوى النقل

لم تكن هذه الإخفاقات استجابات لحمل زائد على مستوى الأعمال، ولم تكن تعني أن مرشح السعة بحد ذاته كان غير صحيح. لقد أشارت إلى أن دفعة قصيرة كانت تضرب طبقة قبول الاتصال في Tomcat/OS قبل أن يتمكن الطلب من الوصول حتى إلى منطق التطبيق.

لتقليل هذا السلوك، تمت زيادة طابور القبول في Tomcat:

- من 50
- إلى 200

هذا مثال على درس عملي: التحكم في التزامن على مستوى التطبيق ضروري، ولكن يجب أيضاً ضبط السعة على مستوى إدخال الشبكة والحاوية بحيث يمكن للطلبات الوصول إلى طبقة التطبيق في المقام الأول.

4.9 التحقق والإثبات

تم إجراء التحقق بثلاث طرق:

أ.

اختبار الإجهاد (Stress testing) كثيف القراءة عبر JMeter

تم بناء اختبار التحميل للضغط على حركة مرور الـ API المصادق عليها بدلاً من سيناريو حالة التسابق في إتمام الطلب من المتطلب الأول. كان هذا مهماً لأن المتطلب الثاني يتعلق بـ إدارة السعة، وليس بصحة المخزون.

ب.

اختبار المرشح الآلي

الملف:

• [src/test/java/com/ecommerce/ecommerce/capacity/RequestCapacityFilterTest.java](#)

يثبت هذا الاختبار:

- عندما تكون السعة ممتلئة، يحصل الطلب المتزامن الثاني على 429
- لا يتم حظر نقاط المراقبة والنقاط العامة بشكل غير صحيح

ج.

استقرار مجموعة الاختبارات الكاملة

استمرت مجموعة الاختبارات الكاملة في النجاح بعد تنفيذ المتطلب الثاني، مما يدل على أن الحماية الإضافية لم تكسر سلوكيات الأعمال الأساسية.

4.10 السلوك بعد الإصلاح

بعد المتطلب الثاني:

- لم يعد التطبيق يعتمد فقط على الإعدادات الافتراضية ذات المستوى الأدنى للتحكم في الحمل
- أصبح عدد الطلبات المتزامنة قيد المعالجة مقيداً بشكل صريح
- أصبحت معالجة الحمل الزائد متعمدة
- استقرت عمليات التشغيل النظيفة المتكررة في نطاق الأخطاء المنخفضة
- أصبح سلوك إدارة الموارد أكثر قابلية للتنبؤ

التفسير الهندسي النهائي ليس أنه لا ينبغي للنظام أبداً رفض أي طلب. التفسير الصحيح هو:

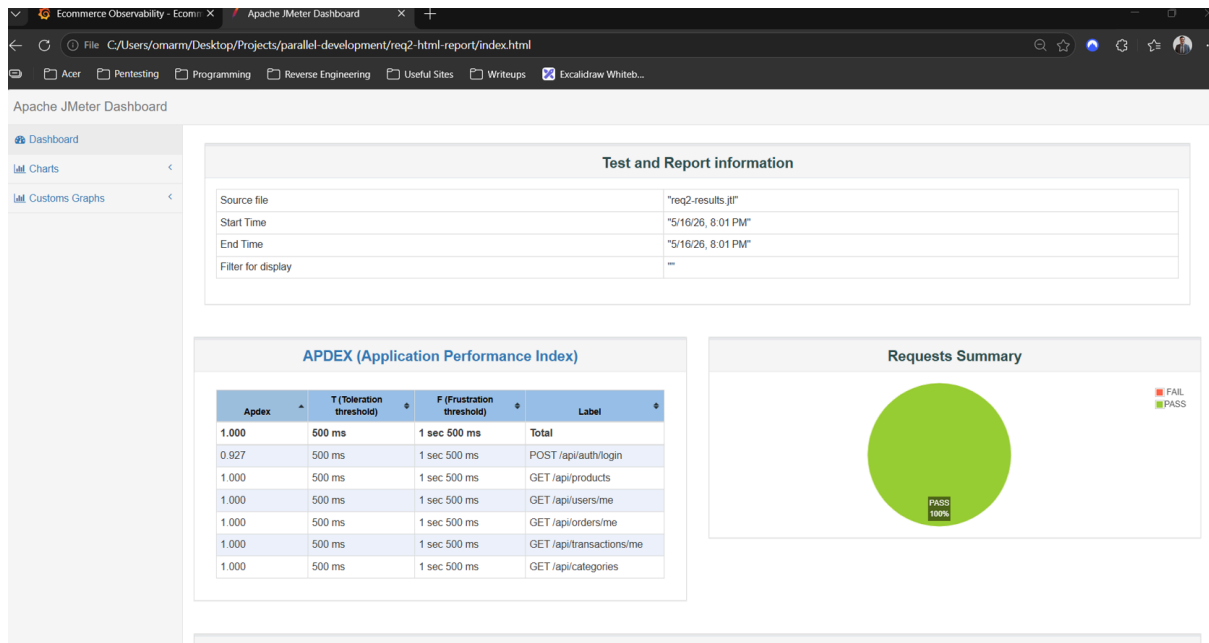
- يجب أن يظل العمل المقبول ضمن السعة
- العمل الزائد يجب أن يتراجع (degrade) بطريقة محكمة
- يجب أن يظل الخادم مستقراً بدلاً من الانهيار

ملخص السلوك بعد الإصلاح

- العمل المتوازي مقيد بحدود
- قبول الطلبات يُدار مركزياً
- معالجة الحمل الزائد صريحة
- تحسن الاستقرار تحت حركة المرور المفاجئة بشكل كبير بعد الضبط

4.11 الإثباتات بالصور بعد حل المشكلة

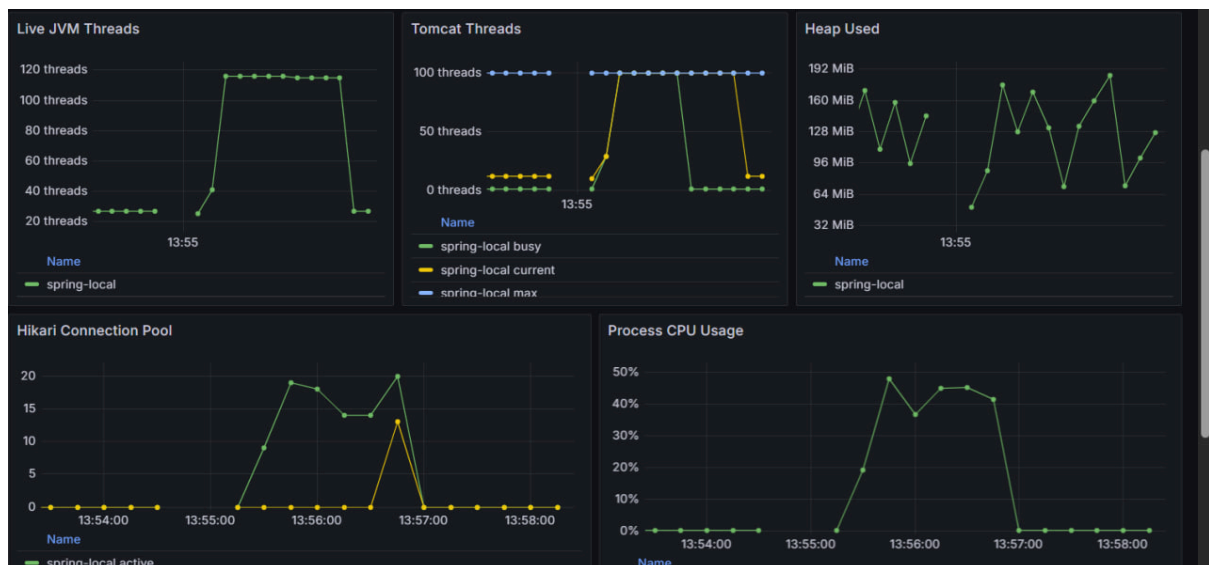
صورة تظهر أن عدد الأخطاء أصبح صفراً بالمئة



ملاحظة مهمة: بعض الأحيان يصبح هناك نسبة أخطاء لا تتجاوز 0.3%

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
POST /api/a...	150	1056	379	2047	312.00	0.00%	13.4/sec	7.40	3.41	564.0
GET /api/pr...	7500	287	24	1048	98.25	0.32%	94.6/sec	59.96	34.54	649.2
GET /api/ca...	7500	284	17	1020	98.56	0.20%	94.8/sec	32.05	34.82	346.1
GET /api/us...	7500	276	14	1061	99.12	0.35%	94.9/sec	41.68	34.66	449.8
GET /api/or...	7500	294	15	1074	102.07	0.23%	95.0/sec	51.34	34.78	553.6
GET /api/tra...	7500	292	14	1209	101.82	0.25%	95.1/sec	53.53	35.37	576.5
TOTAL	37650	290	14	2047	112.85	0.27%	467.5/sec	235.24	171.46	515.2

صورة تظهر استهلاك موارد الجهاز عبر Grafana



5. المعالجة غير المتزامنة Asynchronous Queues

تم تصميم آلية المعالجة غير المتزامنة بهدف فصل زمن تنفيذ المهام التي لا تستوجب انتظار المستخدم لها عن المهام الأساسية التي تتم ضمن التدفق الطبيعي للتطبيق. وتتميز هذه المهام – مثل إرسال إشعارات التأكيد أو إنشاء الفواتير – بأنها لا تحتاج إلى وصول المستخدم إليها في الزمن الحقيقي، مما يسمح بتأخير معالجتها أو إجرائها في خلفية النظام دون التأثير على تجربة المستخدم.

في مشروعنا، قمنا بتطبيق هذا المبدأ الهندسي بشكل خاص عند إرسال رسائل البريد الإلكتروني (Emails)، وذلك لتجنب أي انتظار غير مبرر من قبل العميل أثناء قيامه بعمله على المنصة. فبدلاً من أن ينتظر العميل اكتمال إرسال البريد، يتم وضع مهمة الإرسال في قائمة انتظار وتنفيذها لاحقاً، مما يحسن زمن استجابة التطبيق وسلاسة التفاعل.

لتنفيذ مفهوم قوائم الانتظار غير المتزامنة، اخترنا الأداة RabbitMQ بدلاً من الاعتماد على قوائم الانتظار الداخلية (Memory Queues) التي توفرها Spring Boot. ويعود سبب هذا الاختيار إلى أن Memory Queues تعمل ضمن عقدة خادم واحدة فقط؛ فإذا تعطلت هذه العقدة أو أعيد تشغيلها، تفقد جميع البيانات الموجودة في قوائم الانتظار. أما في المتطلب الخامس من مشروعنا، فقد كان من الضروري توزيع العمل على عدة عُقد لتحقيق التوازن والتوافرية العالية، وهذا يتطلب استخدام منصة خارجية تمتلك ذاكرتها المستقلة ولا تتأثر بحالة أي عقدة منفردة. وقد توافرت جميع هذه الميزات في RabbitMQ، بالإضافة إلى دعمه لآليات التوجيه والمرونة في إدارة الرسائل.

تم ربط RabbitMQ بثلاث عمليات أساسية داخل التطبيق، وتم إنشاء قائمة انتظار (queue) خاصة بكل عملية، وهي:

1. عملية إتمام الإيداع (deposit) - حيث يتم إرسال إشعار بنجاح العملية.
2. عملية إتمام إنشاء الطلب (order) - لتأكيد الطلب ومعالجة الفاتورة المرتبطة به.
3. عملية طلب تقرير المبيعات - نظراً لأن إنشاء التقرير قد يستغرق وقتاً طويلاً، تم وضعه في قائمة انتظار غير متزامنة لتحرير المستخدم من الانتظار.

وفي الصورة التالية يمكنك الاطلاع على قوائم الانتظار التي تم إنشاؤها بواسطة Spring Boot، والتي تظهر بأسمائها المرتبطة بالمهام الثلاث المذكورة أعلاه.


RabbitMQ 3.13.7 Erlang 26.2.5.16
Refreshed 2026-05-18 22:51:31
Refresh every 5 seconds
Virtual host All
Cluster rabbit@479dd58ac760
User admin Log out

Overview
Connections
Channels
Exchanges

Queues and Streams
Admin

All queues (3)

Pagination

Page 1 of 1 - Filter:

☐ Regex

 Displaying 3 items , page size up to: 100

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	deposit.completed.queue	classic	D	running	0	0	0			
/	order.placed.queue	classic	D	running	0	0	0	0.00/s	0.00/s	0.00/s
/	report.requested.queue	classic	D	running	0	0	0			

Add a new queue

6 . معالجة البيانات الضخمة على دفعات (Batch Processing)

تهدف هذه العملية إلى تحسين أداء المهام التي تتعامل مع كميات كبيرة من البيانات، وذلك من خلال تجزئة البيانات إلى مجموعات أصغر (دفعات) ومعالجة كل دفعة بشكل مستقل. في مشروعنا، تم اعتماد أسلوب تنسيق كل مجموعة من البيانات كدفعة واحدة منفصلة عن الأخرى.

تم توظيف هذه التقنية بشكل أساسي في عملية استخراج تقرير المبيعات اليومية. وقد صُممت آلية المعالجة على مبدأ "كل شيء أو لا شيء" (All-or-Nothing) بالنسبة لمجموعات البيانات، أي أنه إما أن تنجح معالجة جميع الدفعات بشكل كامل، أو تفشل العملية برمتها. ولضمان الموثوقية، تمت إضافة آلية إعادة المحاولة التلقائية، حيث يُعاد تنفيذ معالجة أي دفعة تتعرض لخطأ ثلاث مرات قبل اعتبار العملية فاشلة.

كذلك، تم ربط هذه الآلية بوظيفة مجدولة (Job) تقوم بالخطوات التالية:

- تقسيم البيانات المستخرجة إلى دفعات، بحيث لا تتجاوز كل دفعة 10 سجلات.
- تجميع جميع الدفعات في ملف CSV واحد موحد.
- إرسال الملف الناتج عبر البريد الإلكتروني إلى شخص محدد مسبقاً

ستجد في الصورة التالية مثالاً عن ملف CSV النهائي الذي تم إنشاؤه وإرساله عبر البريد الإلكتروني.

daily_sales_20260518_2010.csv								Open with ▾
	A	B	C	D	E	F	G	
1	Order ID	User ID	User Email	Order Total	Item Count	Sale Date	Processed At	
2		1	omaribrahim.8x@gmail.com	999.99	1	2026-05-18	2026-05-18 20:10:24	
3		2	shopper30@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
4		3	shopper42@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
5		4	shopper41@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
6		5	shopper13@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
7		6	shopper15@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
8		7	shopper31@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
9		8	shopper19@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
10		9	shopper49@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
11		10	shopper35@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
12		11	shopper43@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
13		12	shopper29@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
14		13	shopper46@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
15		14	shopper47@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
16		15	shopper48@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	
17		16	shopper33@example.com	2999.97	1	2026-05-18	2026-05-18 20:10:24	

7. توزيع الاحمال

في عالم الخدمات الرقمية، لا يمكن الاعتماد بشكل كامل على مشغل واحد ونسخة واحدة من التطبيق دون وجود نسخ أخرى احتياطية أو موازية. فمع تزايد عدد المستخدمين بشكل كبير، نصل حتماً إلى نقطة لا تستطيع فيها هذه العقدة المفردة تحمل المزيد من الضغط، مما يؤدي إلى توقفها عن العمل وتعطل الخدمة بالكامل. هذه المشكلة تُعرف بنقطة الفشل الواحدة (Single Point of Failure).

من هذا المنطلق، تم تطوير مفهوم توزيع الأحمال بهدف تخفيف العبء عن أي عقدة واحدة، وتحسين أداء النظام بشكل عام، وضمان توفر الخدمة حتى في أوقات الذروة.

في مشروعنا، قمنا باستخدام أداة Nginx لتحقيق توزيع الأحمال. وقد وقع اختيارنا عليها نظراً لسرعتها العالية وقوة أدائها، إضافة إلى كونها ليس فقط خادم ويب (Web Server) بل أيضاً موزع أحمال متكامل، مما يمنحها مزايا متقدمة تجعلها خياراً متميزاً عن غيرها من الحلول.

كما هو واضح في الصورة التالية، يعمل Nginx بشكل صحيح وبدون أية مشاكل، حيث يتلقى الطلبات ويوزعها بكفاءة مع وجود عقدتين من التطبيق متصلتان ب prometheus :

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system?)
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
```

ecommerce-stack
C:\Users\omarm\Desktop\Projects\parallel-development

View configurations

▶

■

✕

Service	Status	Ports	Actions	Logs
grafana ● grafana/grafana:11 3000:3000	■	⋮		<pre>"Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:25:48 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77283 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:25:53 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77278 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:25:58 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77281 "-" "Prometheus/2.54.1" "-"</pre>
prometheus ● prom/prometheus: 9090:9090	■	⋮		<pre>logger=infra.usagstats t=2026-05-18T20:26:01.118460442Z level=info msg="Usage stats are ready to report"</pre>
redis ● redis:7-alpine 6379:6379	■	⋮		<pre>172.19.0.1 - - [18/May/2026:20:26:03 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77284 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:08 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77278 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:13 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77281 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:18 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77277 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:23 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77284 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:28 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77285 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:33 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77285 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:38 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77284 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:43 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77289 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:48 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77289 "-" "Prometheus/2.54.1" "-"</pre>
rabbitmq ● rabbitmq:3.13-manj 15672:15672 Show all ports (2)	■	⋮		
postgres ● postgres:16-alpine 5432:5432	■	⋮		
nginx ● nginx:1.27-alpine 8080:80	■	⋮		
app1 ● ecommerce-stack-s	■	⋮		
app2 ● ecommerce-stack-s	■	⋮		

ecommerce-stack
C:\Users\omarm\Desktop\Projects\parallel-development

View configurations

▶

■

✕

Service	Status	Ports	Actions	Logs
grafana ● grafana/grafana:11 3000:3000	■	⋮		<pre>"Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:25:48 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77283 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:25:53 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77278 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:25:58 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77281 "-" "Prometheus/2.54.1" "-"</pre>
prometheus ● prom/prometheus: 9090:9090	■	⋮		<pre>logger=infra.usagstats t=2026-05-18T20:26:01.118460442Z level=info msg="Usage stats are ready to report"</pre>
redis ● redis:7-alpine 6379:6379	■	⋮		<pre>172.19.0.1 - - [18/May/2026:20:26:03 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77284 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:08 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77278 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:13 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77281 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:18 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77277 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:23 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77284 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:26:28 +0000] "GET /actuator/prometheus HTTP/1.1" 200 77285 "-" "Prometheus/2.54.1" "-" 172.19.0.1 - - [18/May/2026:20:2</pre>

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
prometheus (1/1 up)					
http://prometheus:9090/metrics	UP	instance="prometheus:9090" job="prometheus"	2.714s ago	7.616ms	
spring-cluster (2/2 up)					
http://app2:8080/actuator/prometheus	UP	environment="docker-cluster" instance="app2:8080" job="spring-cluster"	4.812s ago	9.077ms	
http://app1:8080/actuator/prometheus	UP	environment="docker-cluster" instance="app1:8080" job="spring-cluster"	4.664s ago	9.232ms	
spring-local (1/1 up)					
http://host.docker.internal:8080/actuator/prometheus	UP	environment="local-host" instance="host.docker.internal:8080" job="spring-local"	2.741s ago	24.956ms	

تم ربط Nginx بنسختين من التطبيق من خلال ملف `compose.yml` الموجود في المشروع. وقد اعتمدنا في توزيع الطلبات على خوارزمية Round-robin، والتي تعد من أكثر الخوارزميات وضوحاً وسهولة في الفهم. تعمل هذه الخوارزمية على إرسال الطلبات بالتناوب: الطلب الأول يُرسل إلى العقدة الأولى، والثاني إلى العقدة الثانية، والثالث إلى العقدة الأولى، وهكذا.

لماذا تم اختيار خوارزمية Round-robin تحديداً؟

لأن مشروعنا تم بناؤه بالاعتماد على قاعدة بيانات مشتركة بين جميع النسخ، مما يعني أن جلسة المستخدم (Session) لا تعتمد على عقدة معينة، وبالتالي لسنا بحاجة إلى استخدام خوارزمية ip-hash التي توجه طلبات نفس المستخدم دائماً إلى نفس العقدة.

نظراً لأن النسختين من التطبيق تعملان على جهاز واحد، فإن أداء العقدتين متطابق تقريباً، مما لا يستدعي توزيع الأحمال بناءً على فحص الحالة (Health Check) أو وزن العقدة (Weighted balancing)، حيث لا توجد فروقات كبيرة في الحمل أو الاستجابة بينهما.

ملاحظة هامة:

كما هو موضح في الصور السابقة، فإن نظام Nginx يعمل بشكل طبيعي ودون أية مشاكل. ومن المهم أن ندرك أنه لا يمكن إثبات أن نظام توزيع الأحمال يعمل بشكل صحيح إلا من خلال التأكد من أن Nginx نفسه يعمل ويتلقى الطلبات ويوزعها بين العقد، وهو ما تم التحقق منه بنجاح في بيئة المشروع.